

Chapter 2. The unified log

This chapter covers

- Understanding the key attributes of a unified log
- Modeling events using JSON
- Setting up Apache Kafka, a unified log
- Sending events to Kafka and reading them from Kafka

The previous chapter introduced the idea of *events* and *continuous streams of events* and showed that many familiar software platforms and tools have event-oriented underpinnings. We recapped the history of business intelligence and data analytics, before introducing an event-centric data processing architecture, built around something called a unified log. We started to show the *why* of the unified log with some use cases but stopped short of explaining *what* a unified log is.

In this chapter, we will start to get hands-on with unified log technology. We will take a simple Java application and show how to update it to send events to a unified log. Understanding the theory and design of unified logs is important too, so we'll introduce the core attributes of the unified log first.

We have a few unified log implementations to choose from. We'll pick Apache Kafka, an open source, self-hosted unified log to get us started. With the scene set, we will code up our simple Java application, start configuring Kafka, and then code the integration between our app and Kafka. This process has a few discrete steps:

1. Defining a simple format for our events
2. Setting up and configuring our unified log
3. Writing events into our unified log
4. Reading events from our unified log

2.1. Understanding the anatomy of a unified log

All this talk of a unified log, but what exactly is it? A *unified log* is an append-only, ordered, distributed log that allows a company to centralize its continuous event streams. That's a lot of jargon in one sentence. Let's unpack the salient points in the following subsections.

2.1.1. Unified

What does it mean that our log is *unified*? It means that we have a single deployment of this technology in our company (or division or whatever), with multiple applications sending events to it and reading events from it. The Apache Kafka project (Kafka is a unified log) explains it as follows on its homepage (<https://kafka.apache.org/>):

Kafka is designed to allow a single cluster to serve as the central data backbone for a large organization.

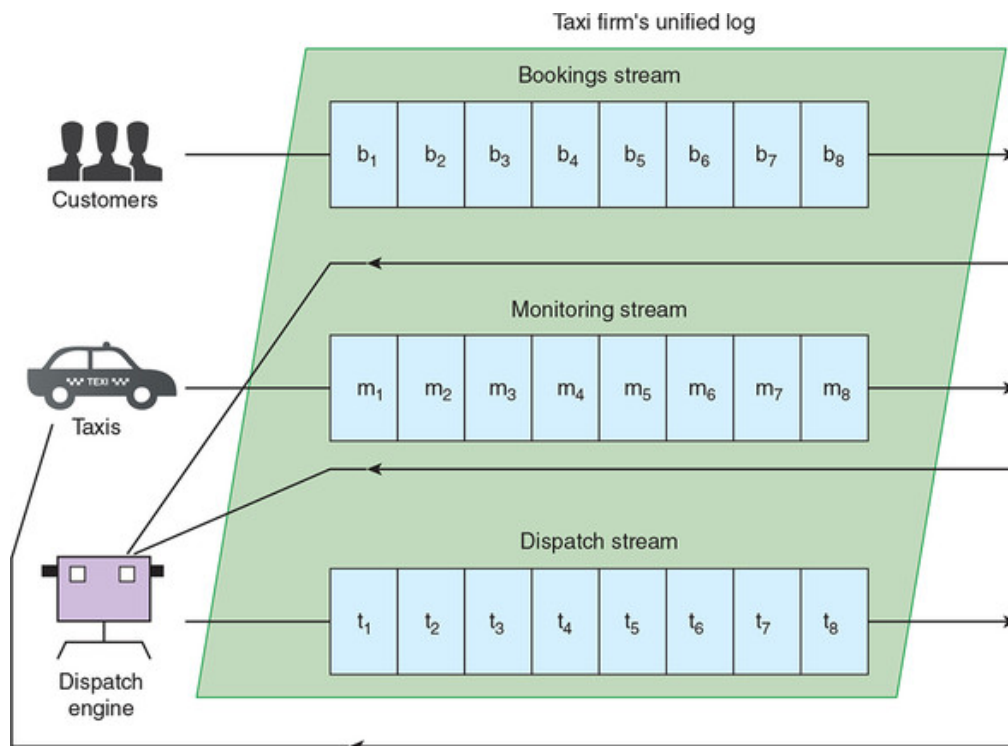
Having a single unified log does *not* mean that all events have to be sent to the same event stream—far from it: our unified log can contain many distinct continuous streams of events. It is up to us to define exactly how we map our business processes and applications onto continuous event streams within our unified log, as we will explore further in [chapter 3](#).

Let’s imagine a metropolitan taxi firm that is embracing the unified log wholeheartedly. Several interesting “actors” are involved in this taxi business:

- Customers booking taxis
- Taxis generating location, speed, and fuel-consumption data
- The Dispatch Engine assigning taxis to customer bookings

[Figure 2.1](#) demonstrates one possible way of architecting this taxi firm around its new unified log implementation. The three streams share the same unified log; there is no reason for them not to. Applications such as the Dispatch Engine can read from two continuous streams of events and write into another stream.

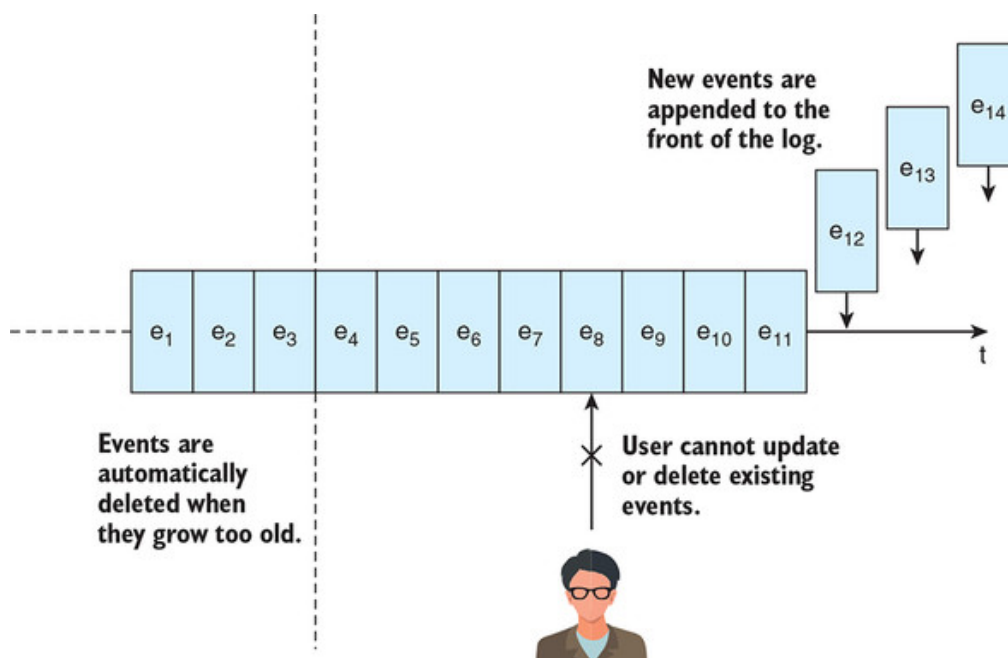
Figure 2.1. The unified log for our taxi firm contains three streams: a taxi-booking stream, a taxi-monitoring stream, and a taxi-dispatching stream.



2.1.2. Append-only

Append-only means that new events are appended to the front of the unified log, but existing events are never updated in place after they're appended. What about deletion? Events are automatically deleted from the unified log when they age beyond a configured time window, but they cannot be deleted in an ad hoc fashion. This is illustrated in [figure 2.2](#).

Figure 2.2. New events are appended to the front of the log, while older events are automatically deleted when they age beyond the time window supported by the unified log. Events already in the unified log cannot be updated or deleted in an ad hoc manner by users.



Being append-only means that it is much easier for your applications to reason about their interactions with the unified log. If your application has read events up to and including event number 10, you know that you will never have to go back and look at events 1 through 10 again.

Of course, being append-only brings its own challenges: if you make a mistake when generating your events, you cannot simply go into the unified log and apply changes to fix those events, as you might in a conventional relational or NoSQL database. But we can compensate for this limitation by carefully modeling our events, building on our understanding of events from [chapter 1](#); we'll look at this in much more detail in [chapter 5](#).

2.1.3. Distributed

Distributed might sound a little confusing: is the log unified, or is it distributed? Actually, it's both! *Distributed* and *unified* are referring to different properties of the log. The log is unified because a single unified log implementation is at the heart of the business, as explained previously in [section 2.1.1](#). The unified log is *distributed* because it lives across a cluster of individual machines.

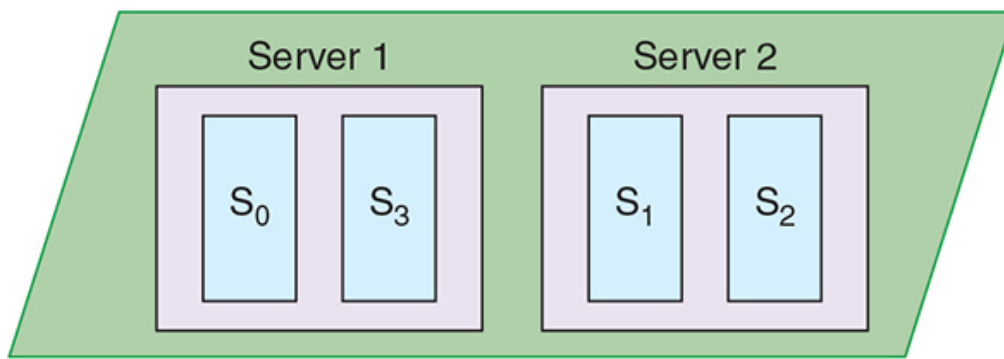
Clustered software tends to be more complex to set up, run, and reason about than software that lives inside one machine. Why do we distribute the log across a cluster? For two main reasons:

- **Scalability**— Having the unified log distributed across a cluster of machines allows us to work with event streams larger than the capacity of any single machine. This is important because any given stream of events (for example, taxi telemetry data from our previous example) could be very large. Distribution across a cluster also makes it easy for each application reading the unified log to be clustered.
- **Durability**— A unified log will replicate all events within the cluster. Without this event distribution, the unified log would be vulnerable to data loss.

To make it easier to work across a cluster of machines, unified logs tend to divide the events in a given event stream into multiple *shards* (sometimes referred to as *partitions*); each shard will be replicated to multiple machines for durability, but one machine will be elected leader to handle all reads and writes. [Figure 2.3](#) depicts this process.

Figure 2.3. Our unified log contains a total of four *shards* (aka *partitions*), split across two physical servers. For the purposes of this diagram, we show each partition only on its leader server. In practice, each partition would be replicated to the other server for failover.

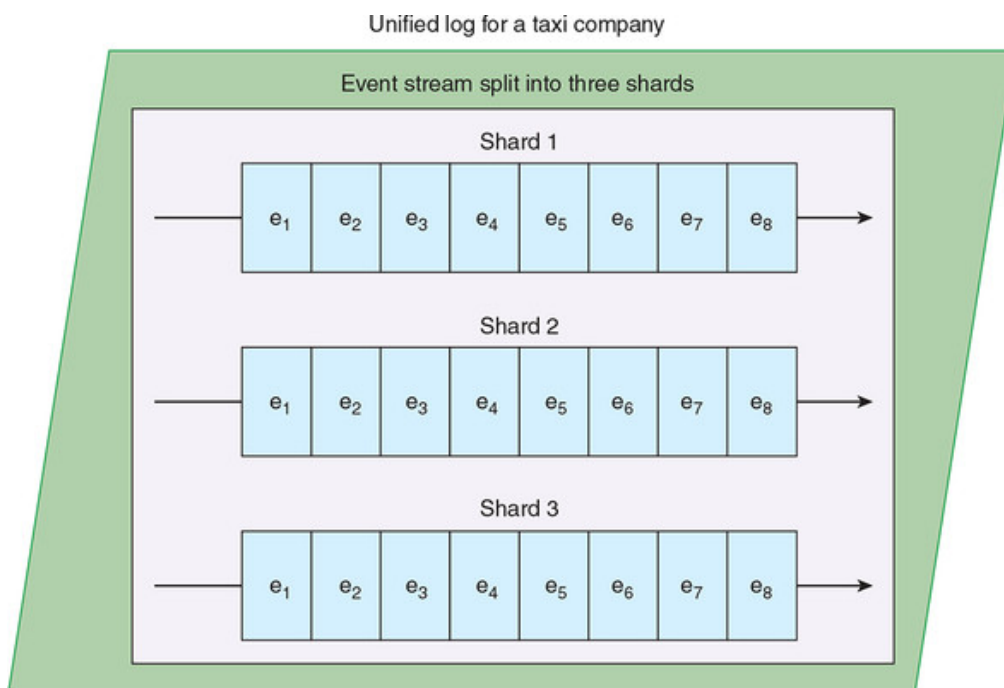
Distributed unified log



2.1.4. Ordered

Ordered means that the unified log gives each event in a shard a sequential ID number (sometimes called the *offset*) that uniquely identifies each message within the shard. Keeping the ordering restricted to the shard keeps things much simpler—because there is no need to maintain and share a global ordering across the whole cluster. [Figure 2.4](#) shows an example of the ordering within a three-shard stream.

Figure 2.4. Our taxi company’s unified log contains a single stream, holding events generated by customers, taxis, and the dispatching engine. In this case, the events are split into three distinct shards. Each shard maintains its own ordering from e1 through to e8.



This ordering gives the unified log much of its power: different applications can each maintain their own cursor position for each shard, telling them which events they have already processed, and thus which events they should process next.

If a unified log did not order events, each consuming application would have to do one of the following:

- **Maintain an event manifest**— Keep a list of all event IDs processed so far and share this with the unified log to determine which events to retrieve next. This approach is conceptually similar to maintaining a manifest of processed files in traditional batch processing.
- **Update or even delete events in the log**— Set some kind of flag against events that have been processed to determine which events to retrieve next. This approach is similar to “popping” a message off a first in, first out (FIFO) queue.

Both of these alternatives would be extremely painful. In the first case, the number of event IDs to keep track of would become hugely unwieldy, reminiscent of the Jorge Luis Borges short story:

In that Empire, the Art of Cartography attained such Perfection that the map of a single Province occupied the entirety of a City, and the map of the Empire, the entirety of a Province.

Jorge Luis Borges, “Del Rigor en la Ciencia” (1946)

The second option would not be much better. We would lose the immutability of our unified log and make it hard for multiple applications to share the stream, or for the sample application to “replay” events it had already processed. And in both situations, the unified log would have to support random access to individual events from consuming applications. So, ordering the log makes a huge amount of sense.

To recap: you have now seen why the unified log is *unified*, why it is *append-only*, why it is *ordered*, and why, indeed, it is *distributed*. Hopefully, this has started to clarify how a unified log is architected, so let’s get started using one.

2.2. Introducing our application

With the basic theory of the unified log set out, let’s begin to put the theory to work! In each part of this book, we will be working with a fictitious company that wants to implement a unified log across its business. To keep things interesting, we will choose a company in a different sector each time. Let’s start with a sector that almost all of us have experienced, at least as customers: e-commerce.

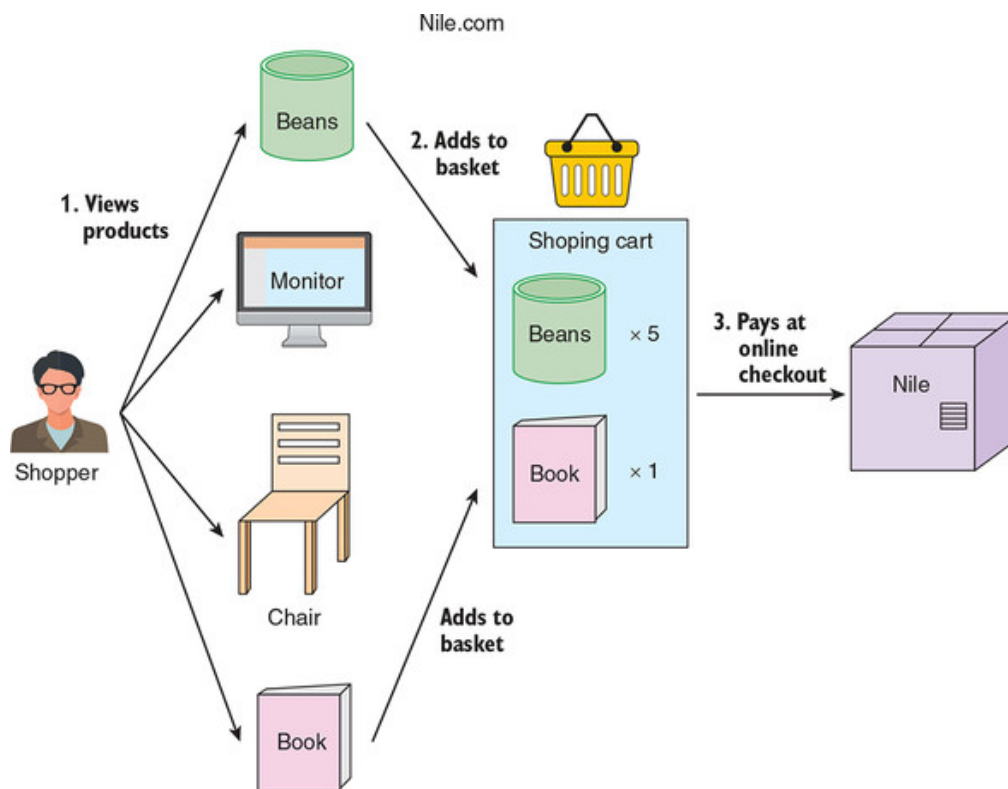
Imagine that we work for a sells-everything e-commerce website; let’s call it *Nile*. The management team at Nile wants the company to become much more dynamic and responsive: Nile’s analysts should have access to up-to-the-minute sales data, and Nile’s systems should react to customer

behavior in a timely fashion. As you will see in this part of the book, we can meet their requirements by implementing a unified log.

2.2.1. Identifying our key events

Online shoppers browse products on the Nile website, sometimes adding products to their shopping cart, and sometimes then going on to buy those products through the online checkout. Visitors can do plenty of other things on the website, but Nile’s executives and analysts care most about this Viewing through Buying workflow. [Figure 2.5](#) shows a typical shopper (albeit with somewhat eclectic shopping habits) going through this workflow.

Figure 2.5. A shopper views four products on the Nile website before adding two of those products to the shopping cart. Finally, the shopper checks out and pays for those items.



Even though our standard definition of an event as *subject-verb-object* will be described in section 2.1.7, we can already identify three discrete events in this Viewing through Buying workflow:

1. *Shopper* views *product* at *time*—Occurs every time the shopper views a product, whether on a product’s dedicated Detail page, or on a general Catalog page that happens to include the product.
2. *Shopper* adds *item* to *cart* at *time*—Occurs whenever the shopper adds one of those products to the shopping basket. A product is added to the basket with a quantity of one or more attached.

3. *Shopper places order at time*—Occurs when the shopper checks out, paying for the items in the shopping basket.

To avoid complicating our lives later, this part of the book keeps this workflow deliberately simple, steering clear of more-complex interactions, such as the shopper adjusting the quantity of an item in the shopping basket, or removing an item from the basket at checkout. But no matter: the preceding three events represent the essence of the shopping experience at Nile.

2.2.2. Unified log, e-commerce style

Nile wants to introduce Apache Kafka (<https://kafka.apache.org>) to implement a unified log across the business. Future chapters cover Kafka in much more detail. For now, it's important to understand only that Kafka is an open source unified log technology that runs on the Java virtual machine (JVM).

We can define an initial event stream (aka *Kafka topic*) to record the events generated by shoppers. Let's call this stream `raw-events`. In [figure 2.6](#), you can see our three types of events feeding into the stream. To make this a little more realistic, we've made a distinction based on how the events are captured:

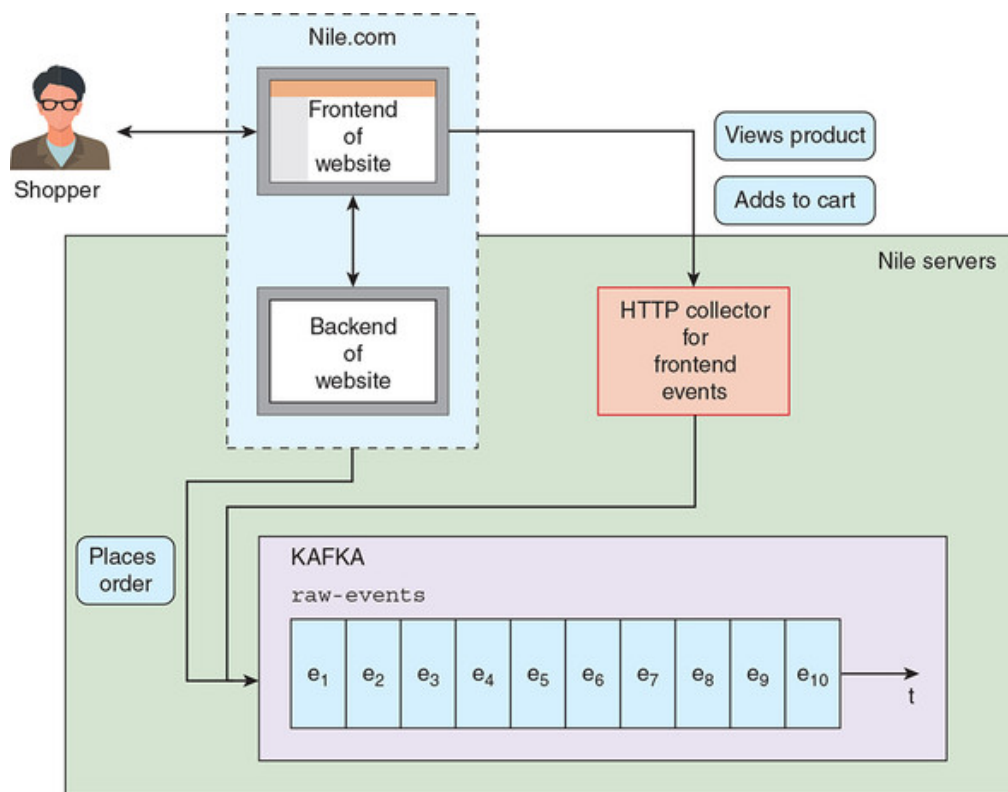
- **Browser-generated events**— The *Shopper views product* and *Shopper adds item to basket* events occur in the user's browser. Typically, there would be some JavaScript code to send the event to an HTTP-based event collector, which would in turn be tasked with writing the event to the unified log.^{[1](#)}

[1](#)

This JavaScript code can be found at <https://github.com/snowplow/snowplow-javascript-tracker>.

- **Server-side events**— A valid *Shopper places order* event is confirmed server-side only after the payment processing has completed. It is the responsibility of the web server to write this event to Kafka.

Figure 2.6. Our three types of event are flowing through, into the `raw-events` topic in Kafka. Order events are written to Kafka directly from the website's backend; the in-browser events of viewing products and adding products to the cart are written to Kafka via an HTTP server that collects events.



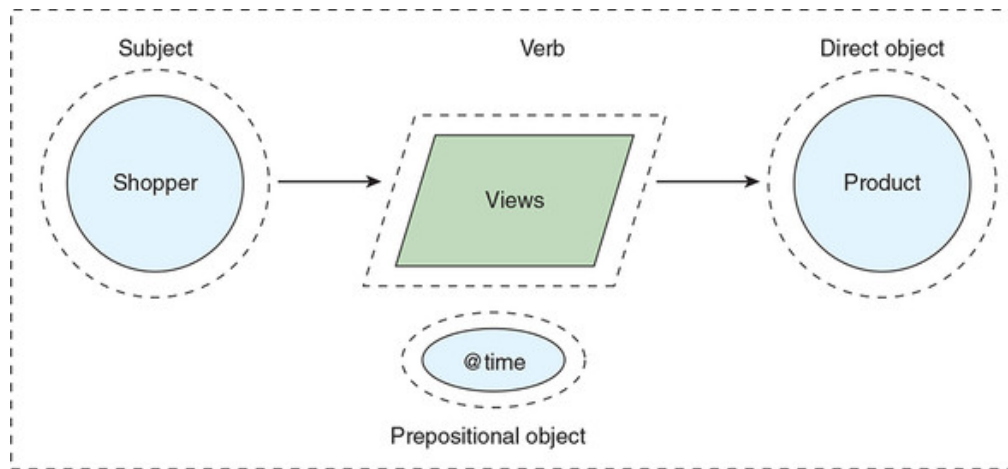
What should our three event types look like? Unified logs don't typically have an opinion on the schema for the events passing through them. Instead they treat each event as a “sealed envelope,” a simple array of bytes to be appended to the log and shared as is with applications when requested. It is up to us to define the internal format of our events—a process we call *modeling* our events.

The rest of this chapter focuses on just a single Nile event type: *Shopper views product at time*. We will come back to the other two Nile event types in the next two chapters.

2.2.3. Modeling our first event

How should we model our first event, *Shopper views product at time*? The secret is to realize that our event already has an inherent structure: it follows the grammatical rules of a sentence in the English language. For those of us who are a little rusty on English language grammar, [figure 2.7](#) maps out the key grammatical components of this event, expressed as a sentence.

Figure 2.7. Our *shopper* (subject) *views* (verb) a *product* (direct object) at *time* (prepositional object).



Let's go through each of these components in turn:

- The “shopper” is the sentence’s *subject*. The subject in a sentence is the entity carrying out the action: *Jane* views an iPad at midday.
- “views” is the sentence’s *verb*, describing the action being done by the *subject*: “Jane *views* an iPad at midday.”
- The “product” being viewed is the *direct object*, or simply *object*. This is the entity to which the action is being done: “Jane views *an iPad* at midday.”
- The time of the event is, strictly speaking, another object—an *indirect object*, or *prepositional object*, where the preposition is “at”: “Jane views an iPad at *midday*.”

We now have a way of breaking our event into its component parts, but so far this description is only human-readable. A computer couldn’t easily parse it. We need a way of formalizing this structure further, ideally into a *data serialization format* that is understandable by humans but also can be parsed by computers.

We have lots of options for serializing data. For this chapter, we will pick JavaScript Object Notation (JSON). JSON has the attractive property of being easily written and read by both people and machines. Many, if not most, developers setting out to model their company’s continuous event streams will start with JSON.

The following listing shows a possible representation for our *Shopper views product at time* event in JSON.

Listing 2.1. shopper_viewed_product.json

```
{  
  "event": "SHOPPER_VIEWED_PRODUCT",    1  
  "shopper": {                          2  
    "id": "123",  
    "name": "Jane",
```

```

        "ipAddress": "70.46.123.145"
    },
    "product": {
        "sku": "aapl-001",
        "name": "iPad"
    },
    "timestamp": "2018-10-15T12:01:35Z"
}

```

- **1 The event type as a simple string with a verb in the simple past tense**
- **2 An object representing the shopper who viewed the product**
- **3 An object representing the product that was viewed**
- **4 When the event occurred as an ISO 8601-compatible timestamp**

Our representation of this event in JSON has four properties:

- `event` holds a string representing the type of event.
- `shopper` represents the person (in this case, a woman named Jane) viewing the product. We have a unique `id` for the shopper, her name and a property called `ipAddress`, which is the IP address of the computer she is browsing on.
- `product` contains the `sku` (stock keeping unit) and `name` of the product, an iPad, being viewed.
- `timestamp` represents the exact time when the shopper viewed the product.

To look at it another way, our event consists of two pieces of *event meta-data* (namely, the `event` and the `timestamp`), and two *business entities* (the `shopper` and the `product`). Now that you understand the specific format of our event in JSON, we need somewhere to send them!

2.3. Setting up our unified log

We are going to send the event stream generated by Nile into a unified log. For this, we're going to pick Apache Kafka. Future chapters cover Kafka in much more detail. For now, it's just important to understand that Kafka is an open source (Apache License 2.0) unified log technology that runs on the JVM.

Be aware that we are going to start up *and leave running* multiple pieces of software in the next subsection. Get ready with a few terminal windows (or a tabbed terminal client if you're lucky).

2.3.1. Downloading and installing Apache Kafka

A Kafka cluster is a powerful piece of technology. But, fortunately, it's simple to set up a cheap-and-cheerful single-node cluster just for testing purposes. First, download Apache Kafka version 2.0.0:

```
http://archive.apache.org/dist/kafka/2.0.0/kafka_2.12-2.0.0.tgz
```

You will have to access that link in a browser. You cannot use `wget` or `curl` to download the file directly. When you have it, un-tar it:

```
$ tar -xzf kafka_2.12-2.0.0.tgz
$ cd kafka_2.12-2.0.0
```

Kafka uses Apache ZooKeeper (<https://zookeeper.apache.org>) for cluster coordination, among other things. Deploying a production-ready ZooKeeper cluster requires care and attention, but fortunately Kafka comes with a helper script to set up a single-node ZooKeeper instance. Run the script like so:

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties
[2018-10-15 23:49:05,185]
  INFO Reading configuration from: config/zookeeper.properties
  (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2018-10-15 23:49:05,190] INFO
  autopurge.snapRetainCount set to 3
  (org.apache.zookeeper.server.DataDirCleanupManager)
[2018-10-15 23:49:05,191] INFO
  autopurge.purgeInterval set to 0
  (org.apache.zookeeper.server.DataDirCleanupManager)
...
[2018-10-15 23:49:05,269] INFO
  minSessionTimeout set to -1 (org.apache.zookeeper.server.ZooKeeperServ
[2018-10-15 23:49:05,270] INFO
  maxSessionTimeout set to -1 (org.apache.zookeeper.server.ZooKeeperServ
[2018-10-15 23:49:05,307] INFO
  binding to port 0.0.0.0/0.0.0.0:2181
  (org.apache.zookeeper.server.NIOServerCnxnFactory)
```

Now we are ready to start Kafka in a second terminal:

```
$ bin/kafka-server-start.sh config/server.properties
[2018-10-15 23:52:05,332] INFO Registered
  kafka:type=kafka.Log4jController MBean
  (kafka.utils.Log4jControllerRegistration$)
[2018-10-15 23:52:05,374] INFO starting (kafka.server.KafkaServer)
[2018-10-15 23:52:05,375] INFO
  Connecting to zookeeper on localhost:2181 (kafka.server.KafkaServer)
```

```
...
[2018-10-15 23:52:06,293] INFO
Kafka version : 2.0.0 (org.apache.kafka.common.utils.AppInfoParser)
[2018-10-15 23:52:06,337] INFO
Kafka commitId : 3402a8361b734732
      (org.apache.kafka.common.utils.AppInfoParser)
[2018-10-15 23:52:06,411] INFO
[KafkaServer id=0] started (kafka.server.KafkaServer)
```

Great, we now have both ZooKeeper and Kafka running. Our bosses at Nile will be pleased.

2.3.2. Creating our stream

Kafka doesn't use our exact language of continuous event streams. Instead, Kafka producers and consumers interact with *topics*; you might remember the language of topics from our publish/subscribe example with the NSQ message queue in [chapter 1](#).

Let's create a new topic in Kafka called `raw-events`:

```
$ bin/kafka-topics.sh --create --topic raw-events \
  --zookeeper localhost:2181 --replication-factor 1 --partitions 1
Created topic "raw-events".
```

Let's briefly go through the second line of arguments:

- The `--zookeeper` argument tells Kafka where to find the ZooKeeper that is keeping track of our Kafka setup.
- The `--replication-factor` of 1 means that the events in our topic will not be replicated to another server. In a production system, we would increase the replication factor so that we can continue processing in the face of server failures.
- The `--partitions` setting determines how many shards we want in our event stream. One partition is plenty for our testing.

We can see our new topic if we run the `list` command:

```
$ bin/kafka-topics.sh --list --zookeeper localhost:2181
raw-events
```

If you don't see `raw-events` listed or get some kind of Connection Refused error, go back to [section 2.3.1](#) and run through the setup steps in the exact same order again.

2.3.3. Sending and receiving events

Now we are ready to send our first events into the `raw-events` topic in Nile's unified log in Kafka. We can do this at the command line with a simple producer script. Let's start it running:

```
$ bin/kafka-console-producer.sh --topic raw-events \
  --broker-list localhost:9092

[2018-10-15 00:28:06,166] WARN Property topic is not valid
(kafka.utils.VerifiableProperties)
```

Per the command-line arguments, we will be sending, or *producing*, events to the Kafka topic `raw-events`, which is available from the Kafka broker available on our local server on port 9092. To send in our events, you type them in and press Enter after each one:

```
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "123", "name":
"Jane", "ipAddress": "70.46.123.145" }, "product": { "sku": "aapl-001",
"name": "iPad" }, "timestamp": "2018-10-15T12:01:35Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "456", "name":
Mo", "ipAddress": "89.92.213.32" }, "product": { "sku": "sony-072", "na
"Widescreen TV" }, "timestamp": "2018-10-15T12:03:45Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "789", "name":
"Justin", "ipAddress": "97.107.137.164" }, "product": { "sku": "ms-003"
"name": "XBox One" }, "timestamp": "2018-10-15T12:05:05Z" }
```

Press Ctrl-D to exit. We have sent in three *Shopper views product* events to our Kafka topic. Now let's read the same events out of our unified log, using the Kafka command-line consumer script:

```
$ bin/kafka-console-consumer.sh --topic raw-events --from-beginning \
  --bootstrap-server localhost:9092
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "123",
"name": "Jane", "ipAddress": "70.46.123.145" }, "product": { "sku":
"aapl-001", "name": "iPad" }, "timestamp": "2018-10-15T12:01:35Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "456",
"name": "Mo", "ipAddress": "89.92.213.32" }, "product": { "sku":
"sony-072", "name": "Widescreen TV" }, "timestamp":
"2018-10-15T12:03:45Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "789",
"name": "Justin", "ipAddress": "97.107.137.164" }, "product": {
"sku": "ms-003", "name": "XBox One" }, "timestamp":
"2018-10-15T12:05:05Z" }
```

Success! Let's go through these arguments briefly:

- We are specifying `raw-events` as the topic we want to read or consume events from.
- The argument `--from-beginning` indicates that we want to consume events from the start of the stream onward.
- The `--bootstrap-server` argument tells Kafka where to find the running Kafka broker.

This time, press Ctrl-C to exit. As a final test, let's pretend that Nile has a second application that also wants to consume from the `raw-events` stream. It's a key property of a unified log technology such as Kafka that we can have multiple applications reading from the same event stream at their own pace. Let's simulate this with another call of the consumer script:

```
$ bin/kafka-console-consumer.sh --topic raw-events --from-beginning \
  --bootstrap-server localhost:9092
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "123",
  "name": "Jane", "ipAddress": "70.46.123.145" }, "product": { "sku":
  "aapl-001", "name": "iPad" }, "timestamp": "2018-10-15T12:01:35Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "456",
  "name": "Mo", "ipAddress": "89.92.213.32" }, "product": { "sku":
  "sony-072", "name": "Widescreen TV" }, "timestamp":
  "2018-10-15T12:03:45Z" }
{ "event": "SHOPPER_VIEWED_PRODUCT", "shopper": { "id": "789",
  "name": "Justin", "ipAddress": "97.107.137.164" }, "product": { "sku":
  "ms-003", "name": "XBox One" }, "timestamp": "2018-10-15T12:05:05Z" }
^CConsumed 3 messages
```

Fantastic—our second request to read the `raw-events` topic from the beginning has returned the exact same three events. This helps illustrate the fact that Kafka is serving as a kind of *event stream database*. Compare this to a pub/sub message queue, where a single subscriber reading messages “pops them off the queue,” and they are gone for good.

We can now send a well-structured event stream to Apache Kafka—a simple event stream for Nile's data engineers, analysts, and scientists to work with. In the next chapter, we will start to make this raw event stream even more useful for our coworkers at Nile by performing simple transformations on the events in-stream.

Summary

- A unified log is an append-only, ordered, distributed log that allows a company to centralize its continuous event streams.
- We can generate a continuous stream of events from our applications and send those events into an open source unified log technology such

as Apache Kafka.

- We represented our events in JSON, a widely used data serialization format, by using a simple structure that echoes the English-language grammar structure of our events.
- We created a topic in Kafka to store our Nile events—a *topic* is Kafka-speak for a specific stream of events.
- We wrote, or *produced*, events to our Kafka topic by using a simple command-line script.
- We read, or *consumed*, events from our Kafka topic by using a simple command-line script.
- We can have multiple consumers reading from the same Kafka topic at their own pace, which is a key building block of the new unified log architecture.

[Support](#) [Sign Out](#)